

Applied A.I. Solutions Foundations of Data Management

Lab Exercise 1

Date: Feb 10th, 2025

Prepared by: Gavriel Kirichenko (101119609), Bikash Giri (101575097), Chia-Wei Chang (101570243), Hsi-Teng Liu (101576074), Abdul-Rasaq Omisesan (101571156), Callum Arul (101585383), Friha Hussainyar (101591222), Diparshan Bhattarai (101494737), Sergio Alejandro Medrano Díaz (101446299).

Github: https://github.com/frogcoder/GBC_Fund_Data_Management/tree/main

ERD:  Lucidchart document

PDFD:  Lucidchart document

Industry of Study

The dataset represents the Retail & E-Commerce industry, focusing on the sale and distribution of consumer goods across different regions in the US. This industry involves businesses that sell products to individual consumers and corporate clients, either through physical stores or online platforms.

Entities In the dataset

The main entities in this dataset are:

- Client → (Customer ID, Customer Name)
- Product → (Product ID, Product Name)
- Orders → (Order ID, Order Date)
- Sales → (Quantity, Profit)
- Location → (Country/Region, Region, State, City)

Target Audience of the Reports

The **target audience** for the reports consists of different stakeholders within the company, each requiring specific insights based on their roles. The reports will be tailored to two main groups:

Operational Users and **Executive Users**:

1. Operational Report:

- a. Target audience: Store managers, sales managers
- b. Purpose: Monitor operations, analyze sales trends. Monitoring & Control: Track daily sales and inventory levels.
- c. Frequency: Weekly updates
- d. Publishing Method: BI dashboards, periodic email updates

2. Executive Report:

- a. Target audience: Regional managers, senior executives, finance departments
- b. Purpose: Provide decision support, track business performance, optimize profitability. Profitability Analysis: Assess profit margins across different regions and product categories.
- c. Frequency: Monthly publication
- d. Publishing Method: PowerPoint, PDF reports, BI dashboards

Necessary Assumptions for Ensuring Accurate and Timely Deliverables

- Data Integrity – Assume that the provided sales data is complete and accurate, with no missing key transaction details.
- Data is up-to-date at the end of each report period.
- Sufficient resources are available for analysis and report generation.
- Standardized Reporting – Assume all regions follow a unified calculation standard for comparability.
- There is at most one store in one postal code area.
- The product ID is divided into three parts by the dash character, the first part is the category code, and the second is the sub-category.
- The calculation of profit does not include the cost of shipping.
- When a discount is applied to one order, all items of the order are discounted by the same rate.
- When an order is returned, all items within that order are returned

Information Both Operational and Executive Reports Include

- Region
- state
- Sales
- Profit

Clean/Transform the Data Source for Further Use and Data Analysis

- All orders happened in Burlington, Vermont are missing Postal Code, we fill it in with 5401
- One Product ID can represent more than one product which is unusual, to solve the data integrity problem, we make the “Product ID” and “Product Name” as the Composite primary key.
- Row ID is dropped for it has no practical meaning in reporting.
- The “Returned” Column in the “Returned” tab will be remove for it consists of only “Yes” values.
- Add “Unit Price”, the formula is “Sales / (1 - Discount) / Quantity” rounded to the second decimal place. From the dataset, the calculated unit price of each sale is consistent for each product.
- Add “Unit Cost”, the formula is “(Sales - Profit) / Quantity” rounded to the second decimal place. From the dataset, the calculated unit cost of each sale is consistent for each product.

- Add “Category Code”, extracted from “Product ID”
- Add “Subcategory Code”, extracted from “Product ID”
- Add “Segment Code” as identification key for customer segment.
- Add “Ship Mode Code” as identification key for ship mode.
- Add “State Code” as identification key for state.
- Add “Region Code” as identification key for region.
- Add “City ID” as identification key for city.
- Add “Employee ID” assuming every manager is an employee and has a corresponding ID.

Operational Report Design

This weekly operational report is designed to support tactical decision-making at the team and local levels (region, state, country, store). Its goal is to track sales performance and ensure it is developing as expected.

The report provides detailed internal data on the performance of specific products, gross sales, profits, and discounts applied, helping managers identify trends, optimize pricing strategies, and improve sales efficiency.

Data Included

- Region – Geographic area where the store is located (South, West, Central, East).
- State – The specific state where the store operates.
- City – The city where the store is located.
- Item – The product sold.
- Unit Price (\$) – The price per unit of the product.

Key Performance Indicators (KPIs) & Analysis

- Quantity Sold – Total number of units sold for each product.
- Discount Applied (%) – Total discount percentage applied to each product.
- Gross Sales (\$) – Total revenue generated by a product, category and/or location.
- Profit (\$) – Net profit after deducting costs and discounts.

Executive Report Design

This monthly executive report evaluates the company's overall performance over a given period. It provides key insights to support strategic decision-making at the management level, helping leaders set long-term goals and strategies to drive sustainable growth.

By analyzing sales trends and financial performance, this report enables executives to identify opportunities, mitigate risks, and adjust business strategies to ensure sustained financial success.

Data Included

- Region – The geographic sales region where transactions occurred.
- State – The state where transactions occurred

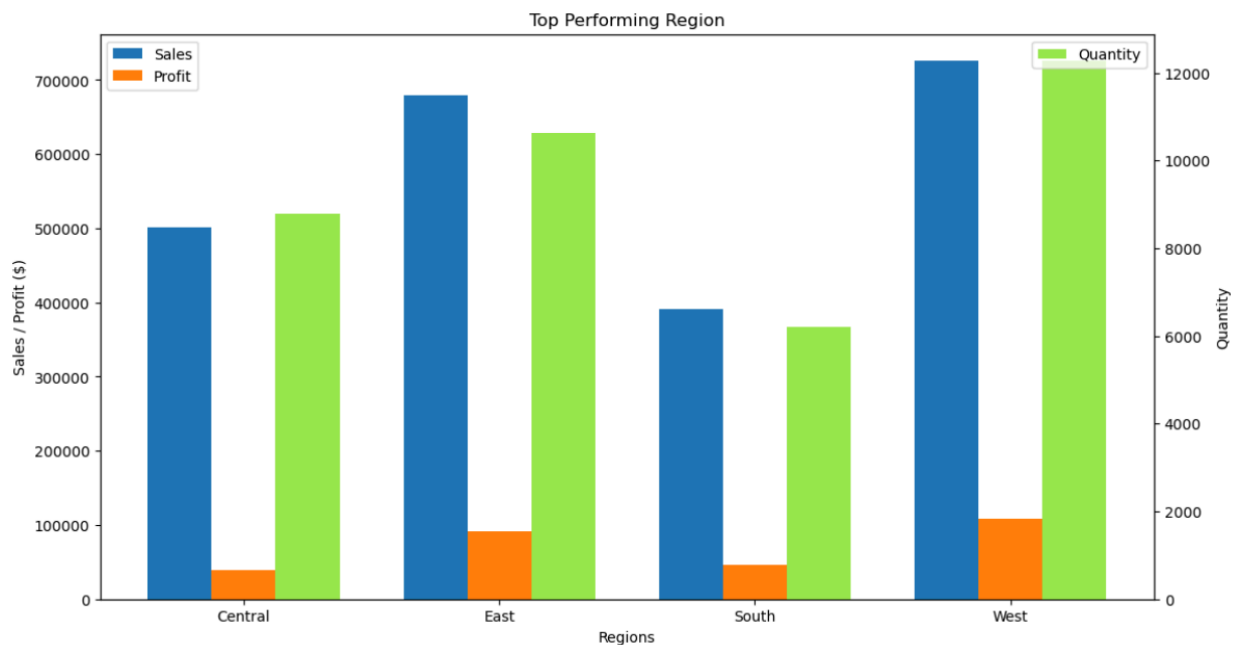
- Period – The period which the products were purchased.
- Quantity Sold, Gross Sales and Profit (\$) – Taken from the operational report.

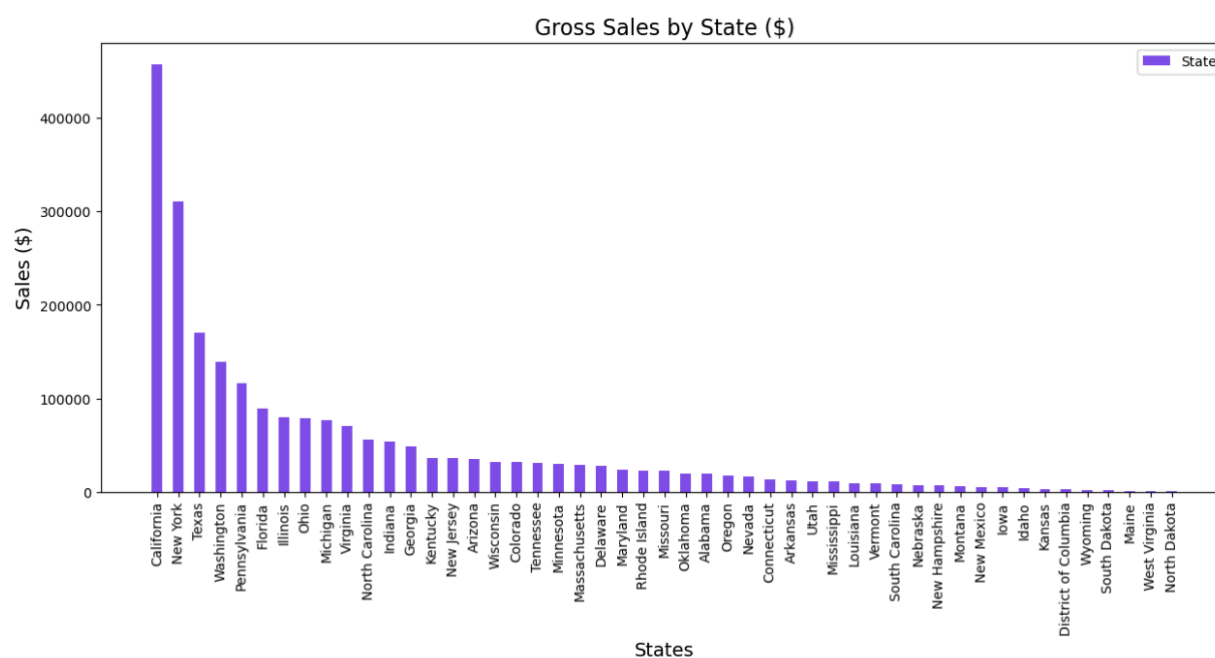
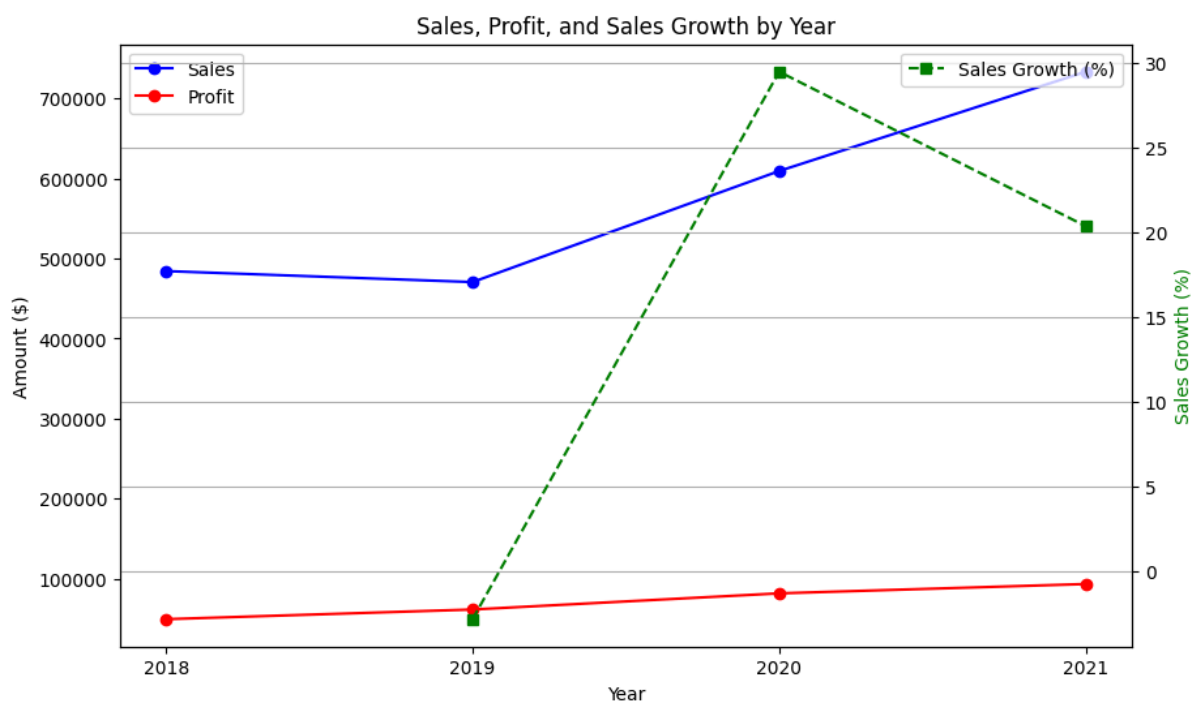
Key Performance Indicators (KPIs) & Analysis

- Previous Monthly Sales (\$) – Total sales amount from the last monthly report.
- Current Monthly Sales (\$) – Total sales amount for the most recent month.
- Monthly Growth (%) – Percentage change in sales compared to the previous month.
- Monthly Growth (\$) – Absolute increase or decrease in sales revenue compared to the previous month.
- Year-over-Year Growth (%) – Percentage change in sales compared to the same month in the previous year.

Regional Performance Comparisons

- Top-Performing Region – The region with the highest sales revenue and growth.
- Lowest-Performing Region – The region with the lowest sales revenue and growth.
- Regional Sales Variance (\$, %) – A comparison of sales performance across different regions.





Data Clean Up and Alignment Jupyter Notebook Source

In [1]:

```
import pandas as pd
```

In [2]:

```
excel_name = "Superstore.xls"
```

In [3]:

```
orders_df = pd.read_excel(excel_name, sheet_name="Orders")
orders_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Row ID          9994 non-null  int64
1   Order ID        9994 non-null  object
2   Order Date      9994 non-null  datetime64[ns]
3   Ship Date       9994 non-null  datetime64[ns]
4   Ship Mode       9994 non-null  object
5   Customer ID     9994 non-null  object
6   Customer Name   9994 non-null  object
7   Segment        9994 non-null  object
8   Country/Region  9994 non-null  object
9   City           9994 non-null  object
10  State          9994 non-null  object
11  Postal Code     9983 non-null  float64
12  Region         9994 non-null  object
13  Product ID     9994 non-null  object
14  Category       9994 non-null  object
15  Sub-Category   9994 non-null  object
16  Product Name    9994 non-null  object
17  Sales          9994 non-null  float64
18  Quantity       9994 non-null  int64
19  Discount       9994 non-null  float64
20  Profit         9994 non-null  float64
dtypes: datetime64[ns](2), float64(4), int64(2), object(13)
memory usage: 1.6+ MB
```

In [4]:

```
people_df = pd.read_excel(excel_name, sheet_name="People")
people_df.head()
```

Out[4]:

	Regional Manager	Region
0	Sadie Pawthorne	West
1	Chuck Magee	East
2	Roxanne Rodriguez	Central

	Regional Manager	Region
3	Fred Suzuki	South

In [5]:

```
returns_df = pd.read_excel(excel_name, sheet_name="Returns")
returns_df.head()
```

Out[5]:

	Returned	Order ID
0	Yes	CA-2018-100762
1	Yes	CA-2018-100762
2	Yes	CA-2018-100762
3	Yes	CA-2018-100762
4	Yes	CA-2018-100867

In [6]:

```
[(n, c.isna().sum()) for n, c in orders_df.items()]
```

Out[6]:

```
[('Row ID', np.int64(0)),
 ('Order ID', np.int64(0)),
 ('Order Date', np.int64(0)),
 ('Ship Date', np.int64(0)),
 ('Ship Mode', np.int64(0)),
 ('Customer ID', np.int64(0)),
 ('Customer Name', np.int64(0)),
 ('Segment', np.int64(0)),
 ('Country/Region', np.int64(0)),
 ('City', np.int64(0)),
 ('State', np.int64(0)),
 ('Postal Code', np.int64(11)),
 ('Region', np.int64(0)),
 ('Product ID', np.int64(0)),
```

```
('Category', np.int64(0)),
('Sub-Category', np.int64(0)),
('Product Name', np.int64(0)),
('Sales', np.int64(0)),
('Quantity', np.int64(0)),
('Discount', np.int64(0)),
('Profit', np.int64(0))]
```

There are 11 missing postal codes

In [7]:

```
orders_df[orders_df["Postal Code"].isna()]["City", "State", "Postal Code"]
```

Out[7]:

	City	State	Postal Code
2234	Burlington	Vermont	NaN
5274	Burlington	Vermont	NaN
8798	Burlington	Vermont	NaN
9146	Burlington	Vermont	NaN
9147	Burlington	Vermont	NaN
9148	Burlington	Vermont	NaN
9386	Burlington	Vermont	NaN
9387	Burlington	Vermont	NaN
9388	Burlington	Vermont	NaN
9389	Burlington	Vermont	NaN
9741	Burlington	Vermont	NaN

In [8]:

```
orders_df.loc[(orders_df["City"] == "Burlington") & (orders_df["State"] == "Vermont"), "Postal Code"]
```

Out[8]:

```
2234  NaN
5274  NaN
8798  NaN
```



```
9146 NaN
9147 NaN
9148 NaN
9386 NaN
9387 NaN
9388 NaN
9389 NaN
9741 NaN
```

```
Name: Postal Code, dtype: float64
```

ALL missing values are postal codes from the same city Burlington, Vermont. Assume the postal code is 05401 and fill in the value

In [9]:

```
orders_df.loc[(orders_df["City"] == "Burlington") & (orders_df["State"] == "Vermont"), "Postal Code"] = 5401
```

Initial Attributes_

In [10]:

```
init_order_columns = set(orders_df.columns)
init_order_columns
```

Out[10]:

```
{'Category',
 'City',
 'Country/Region',
 'Customer ID',
 'Customer Name',
 'Discount',
 'Order Date',
 'Order ID',
 'Postal Code',
 'Product ID',
 'Product Name',
 'Profit',
 'Quantity',
 'Region',
 'Row ID',
 'Sales',
 'Segment',
 'Ship Date',
 'Ship Mode',
 'State',
 'Sub-Category'}
```

In [11]:

```
init_people_columns = set(people_df.columns)
init_people_columns
```

Out[11]:

```
{'Region', 'Regional Manager'}
```

In [12]:

```
init_return_columns = set(returns_df.columns)
init_return_columns
```

Out[12]:

```
{'Order ID', 'Returned'}
```

Remove Row ID for it serves no practical meaning_

In [13]:

```
orders_df.drop("Row ID", axis=1, inplace=True)
```

In [14]:

```
orders_df["Ship Mode"].value_counts()
```

Out[14]:

```
Ship Mode
Standard Class    5968
Second Class      1945
First Class       1538
Same Day          543
Name: count, dtype: int64
```

Four types of shipping mode: Standard, Second Class, First Class, Same Day Assuming expense order from most to least expensive:

```
Same Day
First Class
Second Class
Standard
```

Add Ship Mode Code attribute

In [15]:

```
ship_name_modes = {"Same Day": "0", "First Class": "1", "Second Class": "2", "Standard Class": "S"}
orders_df["Ship Mode Code"] = orders_df["Ship Mode"].map(ship_name_modes)
orders_df["Ship Mode Code"].isna().sum()
```

Out[15]:

```
np.int64(0)
```

In [16]:

```
orders_df["Segment"].value_counts()
```

Out[16]:

```
Segment
Consumer    5191
Corporate    3020
Home Office  1783
Name: count, dtype: int64
```

Add Segment Code attribute

Three types of customer segments:

```
Consumer
Corporate
Home Office
```

In [17]:

```
segment_name_codes = { "Consumer": "S", "Corporate": "C", "Home Office": "O" }
orders_df["Segment Code"] = orders_df["Segment"].map(segment_name_codes)
orders_df["Segment Code"].isna().sum()
```

Out[17]:

```
np.int64(0)
```

In [18]:

```
orders_df["Country/Region"].value_counts()
```

Out[18]:

```
Country/Region
United States  9994
Name: count, dtype: int64
```

All sales are made in the United States

In [19]:

```
orders_df["City"].value_counts()
```

Out[19]:

```
City
New York City  915
Los Angeles    747
Philadelphia   537
San Francisco  510
Seattle        428
...
```

```
Abilene      1
Montebello   1
Kissimmee    1
Danbury      1
Springdale   1
Name: count, Length: 531, dtype: int64
```

In [20]:

```
orders_df["State"].value_counts()
```

Out[20]:

```
State
California      2001
New York         1128
Texas            985
Pennsylvania     587
Washington       506
Illinois         492
Ohio             469
Florida          383
Michigan          255
North Carolina   249
Arizona          224
Virginia         224
Georgia          184
Tennessee        183
Colorado         182
Indiana          149
Kentucky         139
Massachusetts     135
New Jersey       130
Oregon           124
Wisconsin        110
Maryland         105
Delaware         96
Minnesota        89
Connecticut      82
Missouri         66
Oklahoma         66
Alabama          61
Arkansas         60
Rhode Island     56
Utah             53
Mississippi      53
South Carolina   42
Louisiana        42
Nevada           39
Nebraska         38
New Mexico       37
```

```

Iowa                30
New Hampshire       27
Kansas              24
Idaho               21
Montana             15
South Dakota        12
Vermont             11
District of Columbia 10
Maine               8
North Dakota        7
West Virginia       4
Wyoming             1
Name: count, dtype: int64

```

In [21]:

```

# Add state codes
state_name_codes = {
    "Alabama": "AL", "Alaska": "AK", "Arizona": "AZ", "Arkansas": "AR", "California": "CA",
    "Colorado": "CO", "Connecticut": "CT", "Delaware": "DE", "Florida": "FL", "Georgia": "GA",
    "Hawaii": "HI", "Idaho": "ID", "Illinois": "IL", "Indiana": "IN", "Iowa": "IA",
    "Kansas": "KS", "Kentucky": "KY", "Louisiana": "LA", "Maine": "ME", "Maryland": "MD",
    "Massachusetts": "MA", "Michigan": "MI", "Minnesota": "MN", "Mississippi": "MS", "Missouri": "MO",
    "Montana": "MT", "Nebraska": "NE", "Nevada": "NV", "New Hampshire": "NH", "New Jersey": "NJ",
    "New Mexico": "NM", "New York": "NY", "North Carolina": "NC", "North Dakota": "ND", "Ohio": "OH",
    "Oklahoma": "OK", "Oregon": "OR", "Pennsylvania": "PA", "Rhode Island": "RI", "South Carolina": "SC",
    "South Dakota": "SD", "Tennessee": "TN", "Texas": "TX", "Utah": "UT", "Vermont": "VT",
    "Virginia": "VA", "Washington": "WA", "West Virginia": "WV", "Wisconsin": "WI", "Wyoming": "WY",
    "District of Columbia": "DC", "Guam": "GU", "Marshall Islands": "MH", "Northern Mariana Island": "MP", "Puerto
    Rico": "PR", "Virgin Islands": "VI"
}
orders_df["State Code"] = orders_df["State"].map(state_name_codes)
orders_df[orders_df["State Code"].isna()]["State", "State Code"]

```

Out[21]:

	State	State Code

In [22]:

```
orders_df["Region"].value_counts()
```

Out[22]:

```

Region
West      3203
East      2848
Central   2323
South     1620
Name: count, dtype: int64

```

Add Region Code attribute

In [23]:

```
region_name_codes = { "West": "W", "East": "E", "Central": "C", "South": "S" }  
orders_df["Region Code"] = orders_df["Region"].map(region_name_codes)  
orders_df["Region Code"].isna().sum()
```

Out[23]:

```
np.int64(0)
```

In [24]:

```
orders_df["Product ID"].value_counts()
```

Out[24]:

```
Product ID  
OFF-PA-10001970    19  
TEC-AC-10003832    18  
FUR-FU-10004270    16  
FUR-CH-10002647    15  
FUR-CH-10001146    15  
..  
TEC-MA-10004552     1  
TEC-MA-10003589     1  
OFF-AP-10003099     1  
TEC-PH-10002645     1  
OFF-ST-10001627     1  
Name: count, Length: 1862, dtype: int64
```

In [25]:

```
orders_df["Category"].value_counts()
```

Out[25]:

```
Category  
Office Supplies    6026  
Furniture          2121  
Technology         1847  
Name: count, dtype: int64
```

Three product categories:

```
Office Supplies  
Furniture  
Technology
```

In [26]:

```
orders_df["Sub-Category"].value_counts()
```

Out[26]:

```
Sub-Category
Binders      1523
Paper        1370
Furnishings  957
Phones       889
Storage      846
Art          796
Accessories  775
Chairs       617
Appliances   466
Labels       364
Tables       319
Envelopes    254
Bookcases    228
Fasteners    217
Supplies     190
Machines     115
Copiers      68
Name: count, dtype: int64
```

In [27]:

```
orders_df[["Category", "Sub-Category", "Product ID"]].head()
```

Out[27]:

	Category	Sub-Category	Product ID
0	Furniture	Bookcases	FUR-BO-10001798
1	Furniture	Chairs	FUR-CH-10000454
2	Office Supplies	Labels	OFF-LA-10000240
3	Furniture	Tables	FUR-TA-10000577
4	Office Supplies	Storage	OFF-ST-10000760

Product ID starts with category and subcategory code

Add Category Code attribute

In [28]:

```
orders_df["Category Code"] = orders_df["Product ID"].map(lambda p: p.split("-")[0])
orders_df["Category Code"].value_counts()
```

Out[28]:

```
Category Code
OFF    6026
FUR    2121
TEC    1847
Name: count, dtype: int64
```

Add Sub-Category Code attribute

In [29]:

```
orders_df["Sub-Category Code"] = orders_df["Product ID"].map(lambda p: p.split("-")[1])
orders_df["Sub-Category Code"].value_counts()
```

Out[29]:

```
Sub-Category Code
BI    1523
PA    1370
FU     957
PH     889
ST     846
AR     796
AC     775
CH     617
AP     466
LA     364
TA     319
EN     254
BO     228
FA     217
SU     190
MA     115
CO      68
Name: count, dtype: int64
```

Add Employee ID attribute

In [30]:

```
employee_ids = {
    "Sadie Pawthorne": "SP-00001",
    "Chuck Magee": "CM-00002",
    "Roxanne Rodriguez": "RR-00003",
    "Fred Suzuki": "FS-00004"
}
```



```
people_df["Employee ID"] = people_df["Regional Manager"].map(employee_ids)
```

In [31]:

```
orders_df["Discount"].value_counts()
```

Out[31]:

Discount

0.00 4798

0.20 3657

0.70 418

0.80 300

0.30 227

0.40 206

0.60 138

0.10 94

0.50 66

0.15 52

0.32 27

0.45 11

Name: count, dtype: int64

In [32]:

```
orders_df[["Sales", "Quantity", "Discount", "Profit"]].describe()
```

Out[32]:

	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000
mean	229.858001	3.789574	0.156203	28.656896
std	623.245101	2.225110	0.206452	234.260108
min	0.444000	1.000000	0.000000	-6599.978000
25%	17.280000	2.000000	0.000000	1.728750
50%	54.490000	3.000000	0.200000	8.666500
75%	209.940000	5.000000	0.200000	29.364000
max	22638.480000	14.000000	0.800000	8399.976000

Deepest discount is 0.8
There are negative profits

In [33]:

```
returns_df["Returned"].value_counts()
```

Out[33]:

```
Returned
Yes      800
Name: count, dtype: int64
```

The Returned column serves no purpose for it consists of only Yes values

In [34]:

```
returns_df.drop("Returned", axis=1, inplace=True)
```

In [35]:

```
orders_df.duplicated().sum()
```

Out[35]:

```
np.int64(1)
```

Not any order is a duplication

Is all items returned when an order is returned_

In [36]:

```
order_items_counts = orders_df["Order ID"].value_counts()
return_items_counts = returns_df["Order ID"].value_counts()
returned_order_items_counts = pd.DataFrame({ "Order Item Count": order_items_counts, "Return Item Count":
return_items_counts }).dropna()
returned_order_items_counts[returned_order_items_counts["Order Item Count"] !=
returned_order_items_counts["Return Item Count"]]
```

Out[36]:

	Order Item Count	Return Item Count
Order ID		

For each returned order, all items in the order are returned.

Calculating Unit Prices_

In [37]:

```

sales = orders_df["Sales"]
quantities = orders_df["Quantity"]
discounts = orders_df["Discount"]
sales_before_discounts = sales / (1 - orders_df["Discount"])
unit_prices = (sales_before_discounts / quantities).round(2)
orders_df["Unit Price"] = unit_prices

```

In [38]:

```

product_ids = orders_df["Product ID"]
product_prices = pd.DataFrame({"Product ID": orders_df["Product ID"], "Unit Price": unit_prices}).groupby(["Product ID", "Unit Price"]).value_counts()
product_prices

```

Out[38]:

```

Product ID    Unit Price
FUR-BO-10000112  130.98    1
FUR-BO-10000330  120.98    3
FUR-BO-10000362  170.98    5
FUR-BO-10000468   48.58    6
FUR-BO-10000711   70.98    2
..
TEC-PH-10004912   54.95    3
TEC-PH-10004922   66.99    5
TEC-PH-10004924    7.39    3
TEC-PH-10004959  100.49    2
TEC-PH-10004977  195.99    9
Name: count, Length: 1894, dtype: int64

```

In [39]:

```

# By setting the index to only `Product ID`, any product with more than one unit price will cause error.
# This way making sure each product has only one unit price.
# Note. If one product has more than one unit price does not mean it's invalid. It's just indicate the price has changed before
product_prices.reset_index(1)

```

Out[39]:

	Unit Price	count
Product ID		
FUR-BO-1000011 2	130.98	1
FUR-BO-1000033 0	120.98	3
FUR-BO-1000036	170.98	5

	Unit Price	count
Product ID		
2		
FUR-BO-10000468	48.58	6
FUR-BO-10000711	70.98	2
...	...	...
TEC-PH-10004912	54.95	3
TEC-PH-10004922	66.99	5
TEC-PH-10004924	7.39	3
TEC-PH-10004959	100.49	2
TEC-PH-10004977	195.99	9

1894 rows × 2 columns

There is one unit price per product

In [40]:

```
# Find each product's unit cost
profits = orders_df["Profit"]
unit_costs = ((sales - profits) / quantities).round(2)
orders_df["Unit Cost"] = unit_costs
product_costs = orders_df[["Product ID", "Unit Cost"]]
product_costs
```

Out[40]:

	Product ID	Unit Cost
0	FUR-BO-10001798	110.02

	Product ID	Unit Cost
1	FUR-CH-10000454	170.79
2	OFF-LA-10000240	3.87
3	FUR-TA-10000577	268.12
4	OFF-ST-10000760	9.93
...	...	...
9989	FUR-FU-10001889	7.05
9990	FUR-FU-10000747	38.16
9991	TEC-PH-10003645	119.59
9992	OFF-PA-10004041	4.07
9993	OFF-AP-10002684	85.11

9994 rows × 2 columns

In [41]:

Making sure each order has only one kind of discount

```
orders_df[["Product ID", "Discount"]].groupby("Product ID").value_counts().reset_index(1)
```

Out[41]:

	Discount	count
Product ID		
FUR-BO-10000112	0.30	1
FUR-BO-10000330	0.15	2

	Discount	count
Product ID		
FUR-BO-10000330	0.00	1
FUR-BO-10000362	0.00	2
FUR-BO-10000362	0.15	1
...	...	...
TEC-PH-10004959	0.00	1
TEC-PH-10004959	0.20	1
TEC-PH-10004977	0.20	5
TEC-PH-10004977	0.00	3
TEC-PH-10004977	0.40	1

4119 rows × 2 columns

New or removed columns_

In [42]:

```
# Columns added to orders
set(orders_df.columns) - init_order_columns
```

Out[42]:

```
{'Category Code',
 'Region Code',
 'Segment Code',
 'Ship Mode Code',
 'State Code',
 'Sub-Category Code',
 'Unit Cost',
 'Unit Price'}
```

In [43]:

```
# Columns removed from orders
init_order_columns - set(orders_df.columns)
```

Out[43]:

```
{'Row ID'}
```

In [44]:

```
# Columns added to people
set(people_df.columns) - init_people_columns
```

Out[44]:

```
{'Employee ID'}
```

In [45]:

```
# Columns removed from people
init_people_columns - set(people_df.columns)
```

Out[45]:

```
set()
```

In [46]:

```
# Columns added to Returns
set(returns_df.columns) - init_return_columns
```

Out[46]:

```
set()
```

In [47]:

```
# Columns removed from Returned
init_return_columns - set(returns_df.columns)
```

Out[47]:

```
{'Returned'}
```

Exporting CSVs_

In [48]:

```
import os
if not os.path.isdir("data"):
    os.makedirs("data")
```

In [49]:

```
# Export data/categories.csv
category_index = orders_df[["Category Code", "Category"]].value_counts().index
category_index.to_frame(index=False).set_index(["Category Code"]).to_csv("data/categories.csv",
index_label="category_code", header=["category_name"])
```

In [50]:

```
# Export data/subcategories.csv
subcategory_index = orders_df[["Category Code", "Sub-Category Code", "Sub-Category"]].value_counts().index
subcategory_index.to_frame(index=False).set_index(["Category Code", "Sub-Category
Code"]).to_csv("data/subcategories.csv", index_label=["category_code", "subcategory_code"],
header=["subcategory_name"])
```

In [51]:

```
# Export data/employees.csv
people_df[["Employee ID", "Regional Manager"]].set_index("Employee ID").to_csv("data/employees.csv",
index_label=["employee_id"], header=["employee_name"])
```

In [52]:

```
# Export data/ship_modes.csv
orders_df[["Ship Mode", "Ship Mode Code"]].value_counts().index.to_frame().set_index("Ship Mode
Code").to_csv("data/ship_modes.csv", index_label=["ship_mode_code"], header=["ship_mode_name"])
```

In [53]:

```
# Export data/customer_segments.csv
orders_df[["Segment Code", "Segment"]].value_counts().index.to_frame().set_index("Segment
Code").to_csv("data/customer_segments.csv", index_label=["segment_code"], header=["segment_name"])
```

In [54]:

```
# Export data/customers.csv
orders_df[["Customer ID", "Segment Code", "Customer Name"]].value_counts().index.to_frame().set_index("Customer
ID").to_csv("data/customers.csv", index_label=["customer_id"], header=["segment_code", "customer_name"])
```

In [55]:

```
# Export data/regions.csv
people_df["Region Code"] = people_df["Region"].map(region_name_codes)
people_df[["Region Code", "Region", "Employee ID"]].set_index("Region Code").to_csv("data/regions.csv",
index_label=["region_code"], header=["region_name", "manager_employee_id"])
```

In [56]:

```
# Export data/states.csv
orders_df[["State Code", "Region Code", "State"]].value_counts().index.to_frame().set_index("State
Code").to_csv("data/states.csv", index_label=["state_code"], header=["region_code", "state_name"])
```


In [57]:

```
# Export data/cities.csv
cities = orders_df[["State Code", "City"]].value_counts().index.to_frame().sort_index().reset_index(drop=True)
cities.index += 1
cities.index.to_frame()
```

Out[57]:

	0
1	1
2	2
3	3
4	4
5	5
...	...
600	600
601	601
602	602
603	603
604	604

604 rows × 1 columns

In [58]:

```
cities.to_csv("data/cities.csv", index_label="city_id", header=["state_code", "city_name"])
```

In [59]:

```
cities["City ID"] = cities.index.to_frame()
cities
```

Out[59]:

	State Code	City	City ID
1	AL	Auburn	1

	State Code	City	City ID
2	AL	Decatur	2
3	AL	Florence	3
4	AL	Hoover	4
5	AL	Huntsville	5
...	...	...	...
600	WI	Waukesh a	600
601	WI	Wausau	601
602	WI	West Allis	602
603	WV	Wheeling	603
604	WY	Cheyenn e	604

604 rows × 3 columns

In [60]:

```
orders_df["City ID"] = orders_df.merge(cities, on=["State Code", "City"])[["City ID"]]
```

In [61]:

```
# Export data/stores.csv
```

In [62]:

```
orders_df[["Postal Code", "City ID"]].value_counts().index.to_frame().set_index("Postal Code").to_csv("data/stores.csv", index_label="postal_code", header=["city_id"])
```

In [63]:

```
# Export data/products.csv
```

```
products = orders_df[["Product ID", "Category Code", "Sub-Category Code", "Product Name", "Unit Price", "Unit Cost"]].value_counts().index.to_frame().set_index("Product ID")
products.to_csv("data/products.csv", index_label="product_id", header=["category_code", "subcategory_code", "product_name", "unit_price", "unit_cost"])
```

In [64]:

```
# Export data/orders.csv
```

```
orders = orders_df[["Order ID", "Customer ID", "Postal Code", "Order Date", "Discount", "Sales", "Ship Mode Code",  
"Ship Date"]].set_index("Order ID")  
orders.to_csv("data/orders.csv", index_label="order_id", header=["customer_id", "postal_code", "order_date",  
"discount_percentage", "amount_total", "ship_mode_code", "shipping_date"])
```

In [65]:

```
# Export data/order_items.csv
```

```
order_items = orders_df[["Order ID", "Product ID", "Unit Price", "Unit Cost", "Quantity"]]  
order_items.index += 1  
order_items.to_csv("data/order_items.csv", index_label="order_item_id", header=["order_id", "product_id",  
"unit_price", "unit_cost", "quantity"])
```

In [66]:

```
# Export data/order_returns.csv
```

```
returns_df.to_csv("data/order_returns.csv", index=False, header=["order_id"])
```